



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 08

NOMBRE COMPLETO: ALFONSO D'HERNÁN RODRÍGUEZ
ZULUAGA

N° de Cuenta: 321561556

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 21 de Abril del 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Agregar su dado de 8 caras y editar sus normales para que las caras del dado sean iluminadas correctamente.

Código

```
void CrearPiramide_UP()
{
    unsigned int piramide_indices[] = {
        // Base (2 triangles)
        0, 1, 2,
        2, 3, 0,

        // Front face
        4, 5, 6,
        // Right face
        7, 8, 9,
        // Back face
        10, 11, 12,
        // Left face
        13, 14, 15
    };
}
```

```

GLfloat piramide_vertices[] = {
    // x      y      z      S      T      NX      NY      NZ

    // --- BASE (Bottom) --- Normal: [0, -1, 0]
    -0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  0.0f, -1.0f,  0.0f, // 0
     0.5f, -0.5f,  0.5f,  1.0f,  0.0f,  0.0f, -1.0f,  0.0f, // 1
     0.5f, -0.5f, -0.5f,  1.0f,  1.0f,  0.0f, -1.0f,  0.0f, // 2
    -0.5f, -0.5f, -0.5f,  0.0f,  1.0f,  0.0f, -1.0f,  0.0f, // 3

    // --- FRONT FACE --- Normal: [0, 0.5, 0.8] approx
    -0.5f, -0.5f,  0.5f,  0.021f,  0.6511f,  0.0f,  0.44f, -0.89f, // 4
     0.5f, -0.5f,  0.5f,  0.21f,  0.6511f,  0.0f,  0.44f, -0.89f, // 5
     0.0f,  0.5f,  0.0f,  0.13f,  0.988f,  0.0f,  0.44f, -0.89f, // 6 (Apex)

    // --- RIGHT FACE --- Normal: [0.8, 0.5, 0] approx
     0.5f, -0.5f,  0.5f,  0.267f,  0.6511f, -0.89f, 0.44f,  0.0f, // 7
     0.5f, -0.5f, -0.5f,  0.464f,  0.6511f, -0.89f, 0.44f,  0.0f, // 8
     0.0f,  0.5f,  0.0f,  0.36f,  0.988f, -0.89f, 0.44f,  0.0f, // 9 (Apex)

    // --- BACK FACE --- Normal: [0, 0.5, -0.8] approx
     0.5f, -0.5f, -0.5f,  0.519f,  0.6511f,  0.0f,  0.44f,  0.89f, // 10
    -0.5f, -0.5f, -0.5f,  0.71f,  0.6511f,  0.0f,  0.44f,  0.89f, // 11
     0.0f,  0.5f,  0.0f,  0.606f,  0.988f,  0.0f,  0.44f,  0.89f, // 12 (Apex)

    // --- LEFT FACE --- Normal: [-0.8, 0.5, 0] approx
    -0.5f, -0.5f, -0.5f,  0.776f,  0.6511f,  0.89f, 0.44f,  0.0f, // 13
    -0.5f, -0.5f,  0.5f,  0.945f,  0.6511f,  0.89f, 0.44f,  0.0f, // 14
     0.0f,  0.5f,  0.0f,  0.841f,  0.988f,  0.89f, 0.44f,  0.0f, // 15 (Apex)

```

```

void CrearPiramide_DOWN()
{
    unsigned int piramide_indices[] = {
        // Base (2 triangles)
        0, 1, 2,
        2, 3, 0,

        // Front face
        4, 5, 6,
        // Right face
        7, 8, 9,
        // Back face
        10, 11, 12,
        // Left face
        13, 14, 15
    };

    GLfloat piramide_vertices[] = {
        // x      y      z      S      T      NX      NY      NZ

        // --- BASE (Bottom) --- Normal: [0, -1, 0]
        -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 0
        0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 1
        0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 2
        -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 3

        // --- FRONT FACE --- Normal: [0, 0.5, 0.8] approx
        -0.5f, -0.5f, 0.5f, 0.0f, 0.0465f, 0.0f, 0.44f, -0.89f, // 4
        0.5f, -0.5f, 0.5f, 0.196f, 0.0465f, 0.0f, 0.44f, -0.89f, // 5
        0.0f, 0.5f, 0.0f, 0.103f, 0.384f, 0.0f, 0.44f, -0.89f, // 6 (Apex)

        // --- RIGHT FACE --- Normal: [0.8, 0.5, 0] approx
        0.5f, -0.5f, 0.5f, 0.267f, 0.0465f, -0.89f, 0.44f, 0.0f, // 7
        0.5f, -0.5f, -0.5f, 0.464f, 0.0465f, -0.89f, 0.44f, 0.0f, // 8
        0.0f, 0.5f, 0.0f, 0.36f, 0.384f, -0.89f, 0.44f, 0.0f, // 9 (Apex)

        // --- BACK FACE --- Normal: [0, 0.5, -0.8] approx
        0.5f, -0.5f, -0.5f, 0.540f, 0.0232f, 0.0f, 0.44f, 0.89f, // 10
        -0.5f, -0.5f, -0.5f, 0.743f, 0.0465f, 0.0f, 0.44f, 0.89f, // 11
        0.0f, 0.5f, 0.0f, 0.639f, 0.372f, 0.0f, 0.44f, 0.89f, // 12 (Apex)

        // --- LEFT FACE --- Normal: [-0.8, 0.5, 0] approx
        -0.5f, -0.5f, -0.5f, 0.792f, 0.0581f, 0.89f, 0.44f, 0.0f, // 13
        -0.5f, -0.5f, 0.5f, 0.989f, 0.093f, 0.89f, 0.44f, 0.0f, // 14
        0.0f, 0.5f, 0.0f, 0.885f, 0.441f, 0.89f, 0.44f, 0.0f, // 15 (Apex)
    };
}

```

```
// dado de 8 caras
model = glm::mat4(1.0);

model = glm::translate(model, glm::vec3(-0.0f, 20.0f, 3.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.0f, 4.00f, 4.00f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dice_eight.UseTexture();
meshList[5]->RenderMesh();

model = modelaux;
model = glm::translate(model, glm::vec3(-0.0f, -4.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.0f, 4.00f, 4.00f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dice_eight.UseTexture();
meshList[6]->RenderMesh();
```

Ejecución



2.- Apagar con teclado la luz (pointlight) de su lámpara creada para el reporte de la práctica 7.

Código

Main

```
//información al shader de fuentes de iluminación
shaderList[0].SetDirectionalLight(&mainLight);
if (mainWindow.getapagarLampara() == 1) {
    shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
}
else {
    shaderList[0].SetPointLights(pointLights, pointLightCount);
}
```

Window.cpp

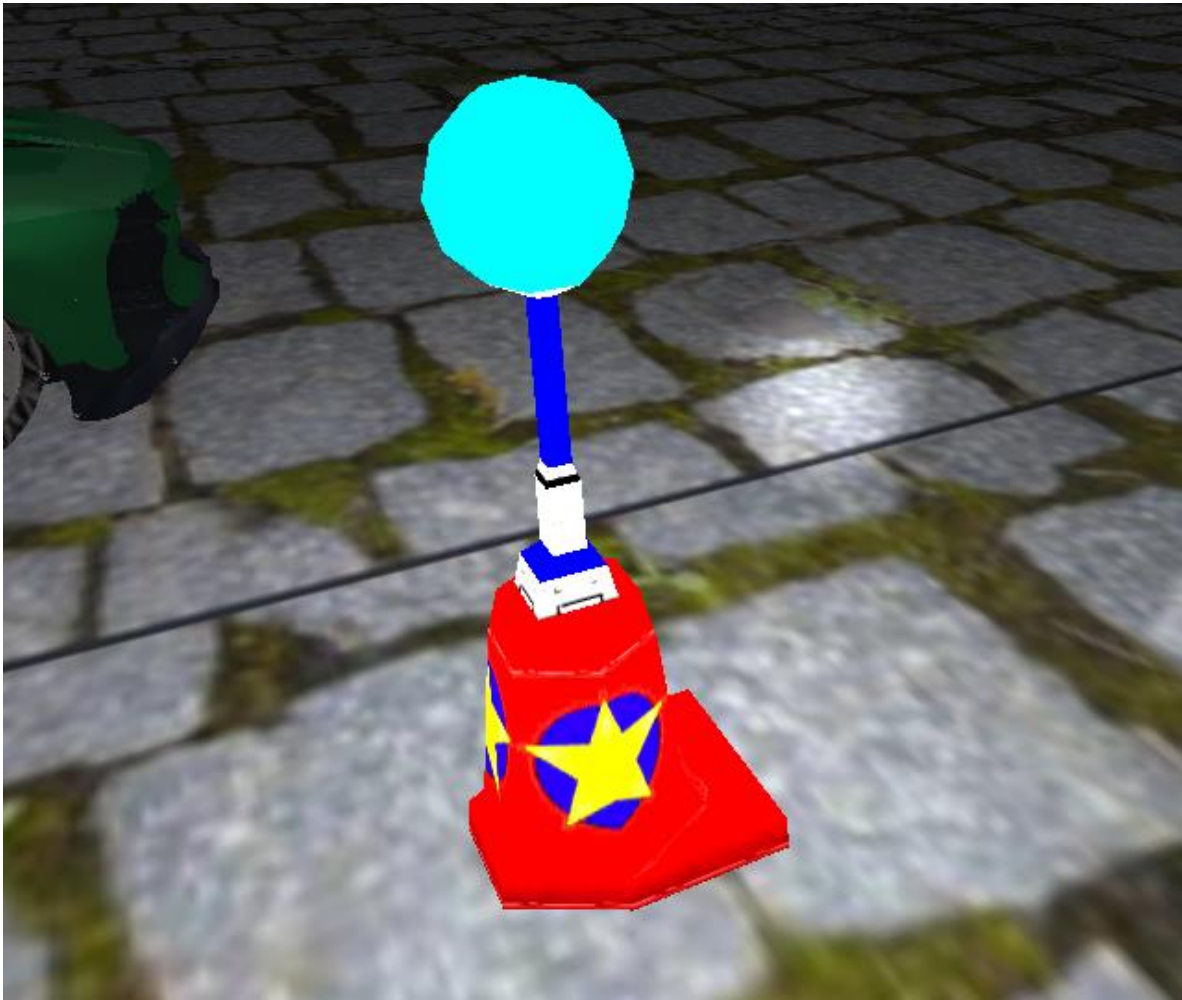
```
if (key == GLFW_KEY_X&& action==GLFW_PRESS) {
    if (theWindow->apagarLampara == 1) {
        theWindow->apagarLampara = 0;
    }
    else {
        theWindow->apagarLampara = 1;
    }
}
```

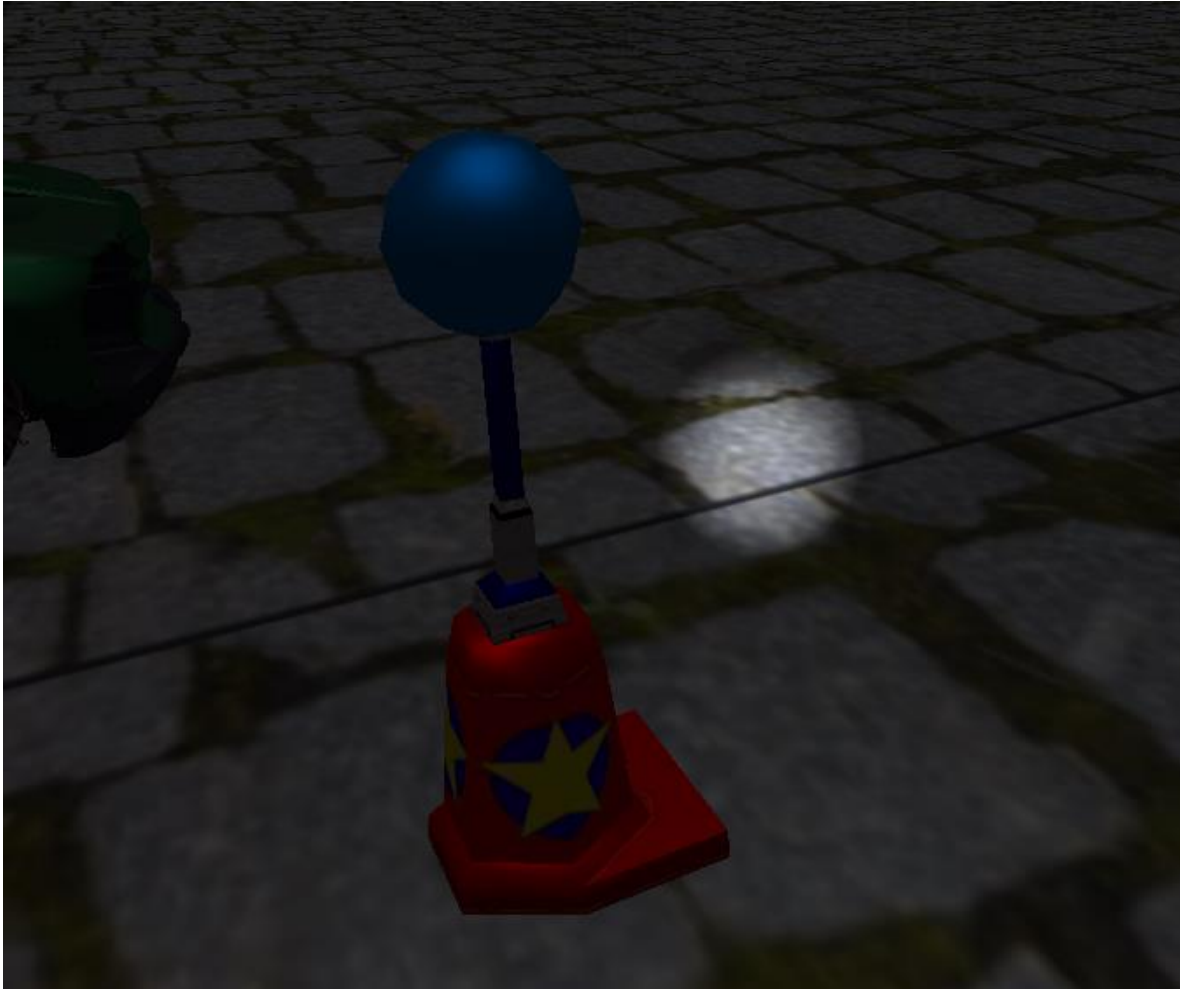
Window.h

```
int getapagarLampara() { return apagarLampara; }

int apagarLampara = 0;
```

Ejecución





3.- Hacer que al presionar una tecla, la luz del faro del coche cicle de color entre rojo, amarillo, verde, cian, azul, magenta

Código

Window.h

```
int colorFaro = 0;  
  
int getColorFaro() { return colorFaro; }
```

Window.cpp


```

if (key == GLFW_KEY_0 && action == GLFW_PRESS) {
    if (theWindow->colorFaro == 5) {
        theWindow->colorFaro = -1;
    }
    theWindow->colorFaro++;
    printf("colorFaro = %d\n", theWindow->colorFaro);
}

```

Spotlight.h

```

void SetLight(GLfloat Red, GLfloat Green, GLfloat Blue);

```

Spotlight.cpp

```

void Spotlight::SetLight(GLfloat Red, GLfloat Green, GLfloat Blue)
{
    color = glm::vec3(Red, Green, Blue);
}

```

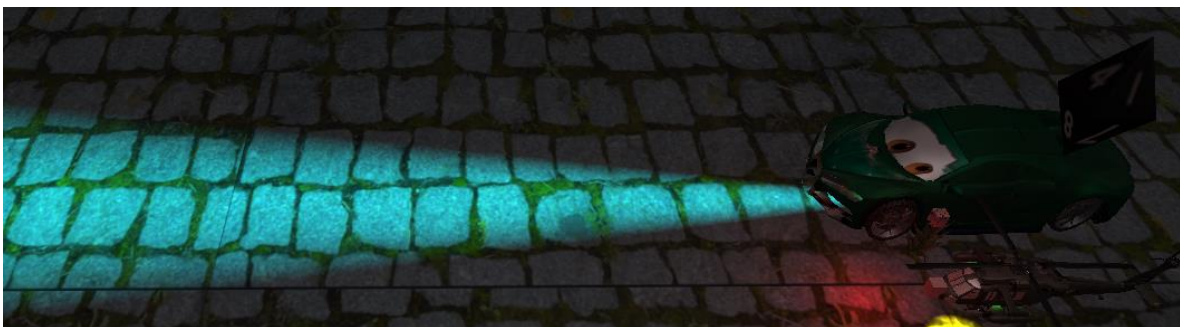
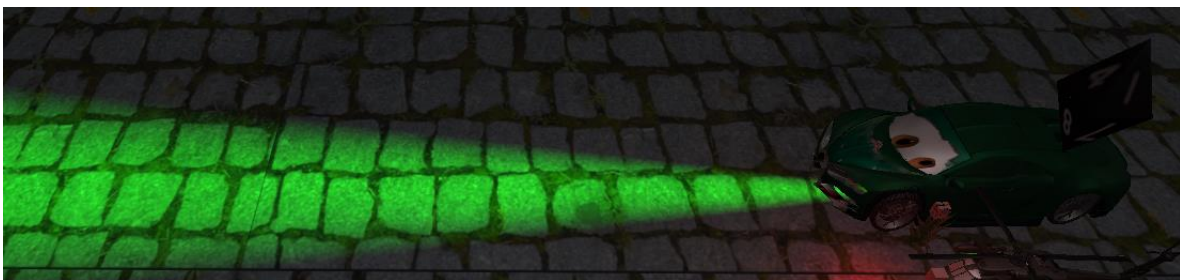
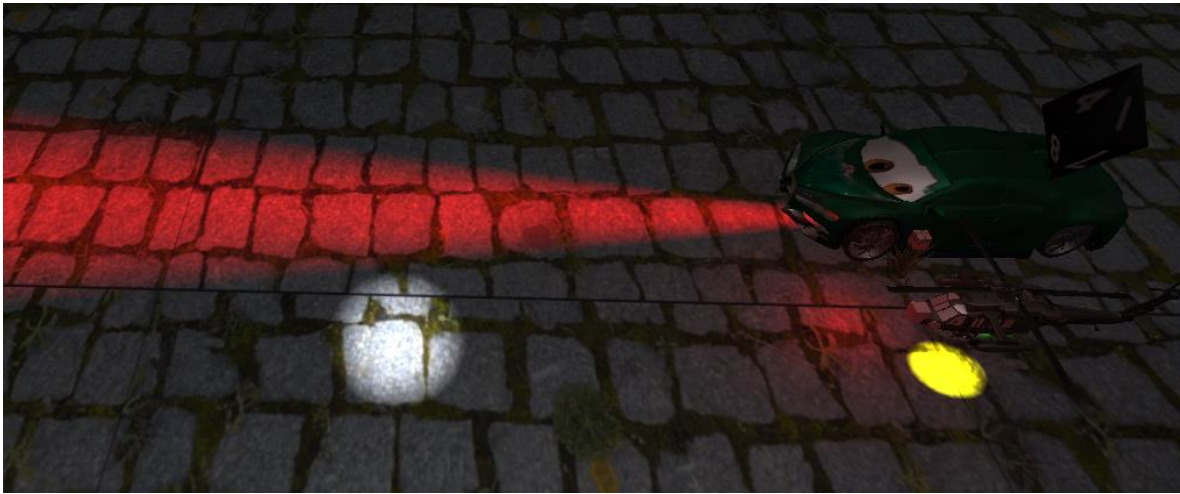
Main

```

if (mainWindow.getColorFaro() == 0) {
    spotLights[3].SetLight(1.0f, 0.0f, 0.0f);
}
else if (mainWindow.getColorFaro() == 1) {
    spotLights[3].SetLight(1.0f, 1.0f, 0.0f);
}
else if (mainWindow.getColorFaro() == 2) {
    spotLights[3].SetLight(0.0f, 1.0f, 0.0f);
}
else if (mainWindow.getColorFaro() == 3) {
    spotLights[3].SetLight(0.0f, 1.0f, 1.0f);
}
else if (mainWindow.getColorFaro() == 4) {
    spotLights[3].SetLight(0.0f, 0.0f, 1.0f);
}
else if (mainWindow.getColorFaro() == 5) {
    spotLights[3].SetLight(1.0f, 0.0f, 1.0f);
}

```

Ejecución





2. Problemas que se tuvieron:

- Tuve un problema a la hora de tratar de implementar el apagado de la luz, pues puse el código correspondiente hasta el final, de manera que las luces no cambiaban en nada. Solo tuve que reacomodar el código.
- A la hora de hacer la función de SetLight, no entendía que había propiedades de herencia para SpotLight, las cuáles fue necesario utilizar para la implementación de los cambios de colores.

3. Conclusión:

- Los ejercicios fueron desafiantes, pero no los encontré particularmente imposibles. Me gustaron.
- La explicación en clase estuvo bien, buen ritmo y se entendió todo.